

Design Principles

Building Microservice Systems

Ali Abrishami

Seyyed Hossein Mirhosseini



SOLID Principles

- Good software systems begin with clean code.
- On the one hand, if the **bricks** aren't well made, the architecture of the **building** doesn't matter much.
- On the other hand, you can make a substantial mess with well-made bricks.
- This is where the ***SOLID principles*** come in.

SOLID Principles (cont.)

- The SOLID principles tell us how to **arrange** our functions and data structures into **classes**, and how those classes should be **interconnected**.
- A class is simply a coupled grouping of functions and data.
- The goal of the principles is the creation of mid-level software structures that:
 - Tolerate change,
 - Are easy to understand,
 - Are the basis of components that can be used in many software systems.

SOLID Principles (cont.)

1. **SRP:** *The Single Responsibility Principle*

Each software module has one, and only one, reason to change.

2. **OCP:** *The Open-Closed Principle*

Allow changing behavior of systems by adding new code, rather than changing existing code.

3. **LSP:** *The Liskov Substitution Principle*

Build software systems from interchangeable parts, those parts must adhere to a contract that allows those parts to be substituted one for another.

SOLID Principles (cont.)

4. **ISP:** *The Interface Segregation Principle*

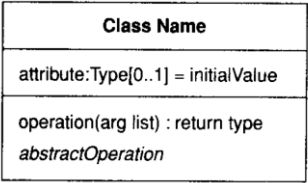
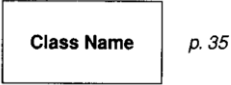
Avoid depending on things that they don't use.

5. **DIP:** *The Dependency Inversion Principle*

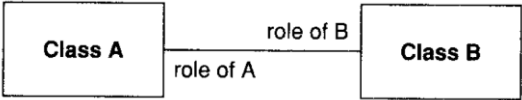
The code that implements high-level policy should not depend on the code that implements low-level details.

Class Diagram

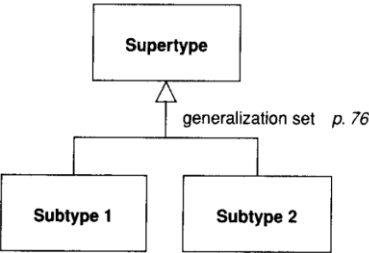
Class



Association *p. 37*

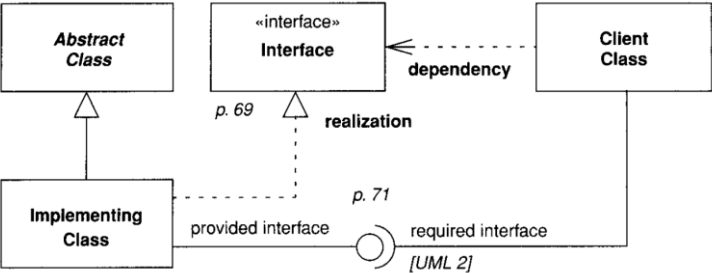


Generalization *p. 45*

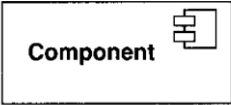


Constraint {name: description} *p. 49*
Keyword «keyword» *p. 65*

Class Diagram



p. 139



Class Diagram

- Class
- Properties
- Associations
- Multiplicity
- Generalization

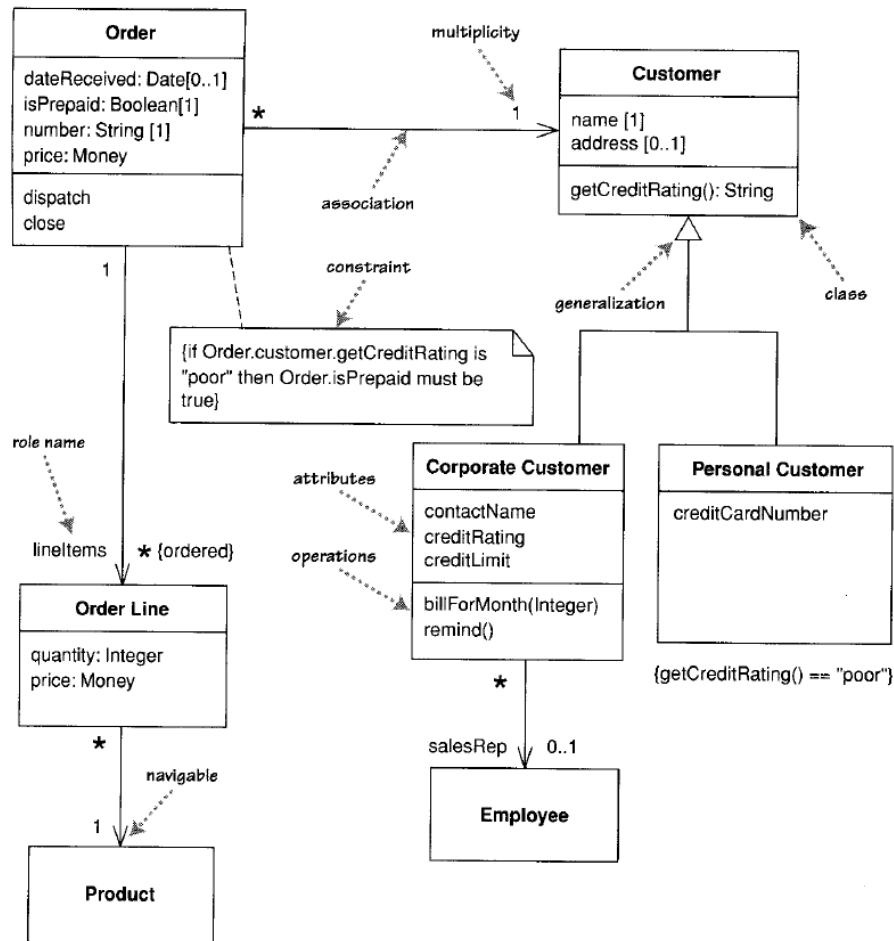
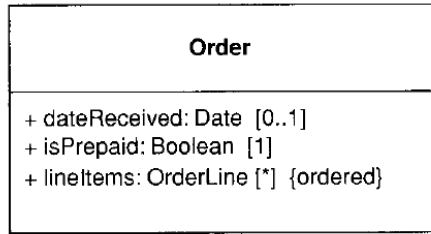
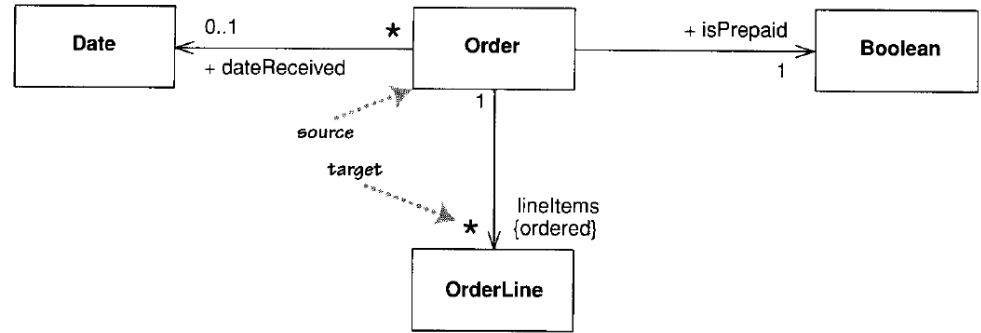


Figure 3.1 A simple class diagram

Attribute vs. Association



Showing properties of an order as attributes



Showing properties of an order as associations

Interface and Abstract Class

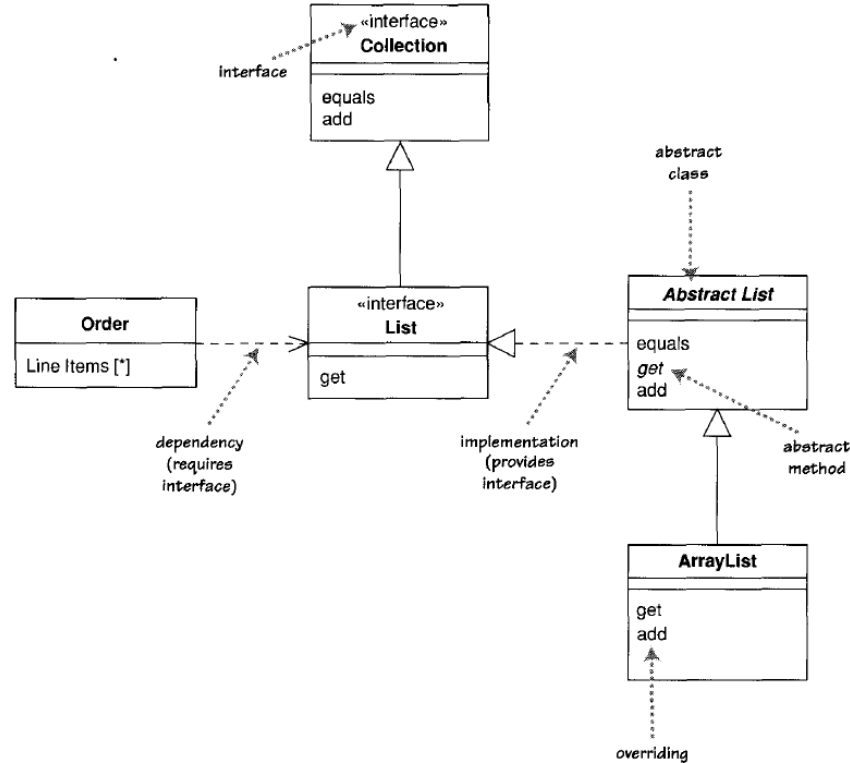


Figure 5.6 A Java example of interfaces and an abstract class

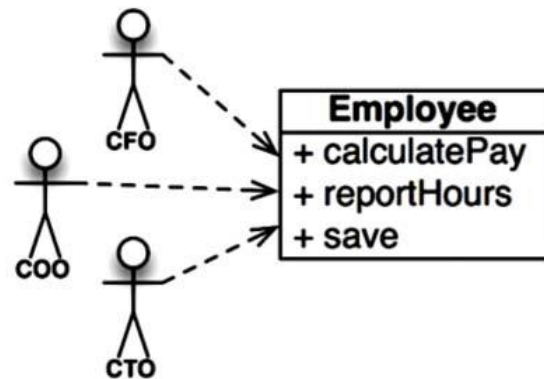
Single Responsibility Principle (SRP)

Single Responsibility Principle

- *A function should do one, and only one, thing.*
- SRP definition:
 - *A module should have one, and only one, reason to **change**.*
 - *A module should be **responsible** to one, and only one, **user** or **stakeholder**.*
 - *A module should be responsible to one, and only one, **actor**.*
- Cohesion is the force that binds together the code responsible to a **single actor**.

Accidental Duplication

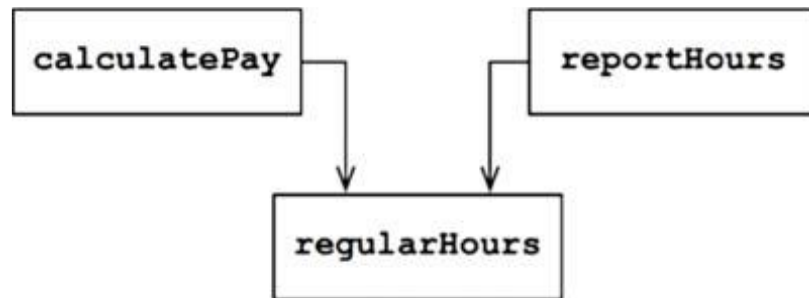
- This class violates the SRP because those three methods are responsible to three very different actors:
 - The *calculatePay()* method is specified by the accounting department.
 - The *reportHours()* method is specified and used by the human resources department.
 - The *save()* method is specified by the DBAs.



The Employee class

Accidental Duplication (cont.)

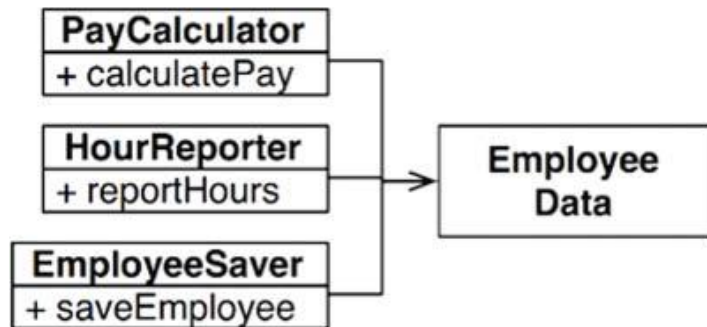
- Suppose that the *calculatePay()* function and the *reportHours()* function share a common algorithm for calculating non-overtime hours.
- Suppose that the CFO's team decides that the way non-overtime hours are calculated needs to be tweaked.



Shared algorithm

Solution

- Move the functions into different classes and separate the data from the functions.
- The three classes share access to *EmployeeData*, which is a simple data structure with no methods.
- Each class holds only the source code necessary for its particular function.



Enhanced design

Conclusion

- We've all seen things like this happen. These problems occur because we put code that **different actors** depend on into close proximity.
- The SRP says to ***separate the code that different actors depend on.***
- The Single Responsibility Principle is about **functions** and **classes**—but it reappears in a different form at two more levels.
- At the level of components, it becomes the Common Closure Principle. At the architectural level, it becomes the Axis of Change responsible for the creation of Architectural Boundaries.

Open Closed Principle (OCP)

Open Closed Principle

- *A software artifact should be **open** for extension but **closed** for modification.*
- The behavior of a software artifact ought to be extendible, without having to modify that artifact.
- A good software architecture would reduce the amount of changed code to the **barest minimum**. Ideally, zero. But, how?
 - By properly separating the things that change for different reasons (the Single Responsibility Principle), and then,
 - Organizing the dependencies between those things properly (the Dependency Inversion Principle).

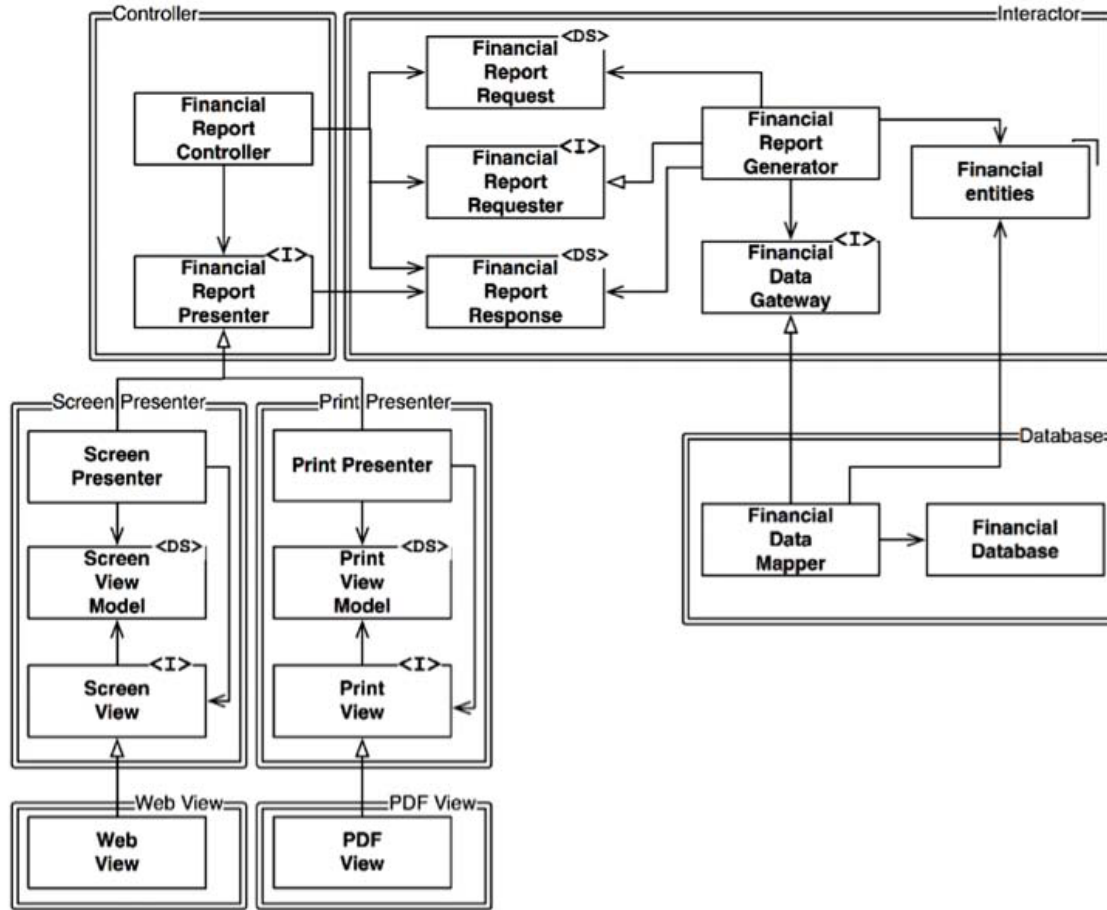
Example

- A financial summary system for showing data on web and also printer.
- We need to organize the source code dependencies to ensure that changes to one of those responsibilities do not cause changes in the other.



Applying the SRP

Example



Partitioning the processes into classes and separating the classes into components

Example

- Notations used:
 - Classes marked with <I> are **interfaces**.
 - Classes marked with <DS> are **data structures**.
 - Open arrowheads are ***using*** relationships.
 - Closed arrowheads are ***implements*** or ***inheritance*** relationships.
- 1) We want to protect the *Controller* from changes in the *Presenters*.
 - 2) We want to protect the *Presenters* from changes in the *Views*.
 - 3) We want to protect the *Interactor* from changes in—well, *anything*.

Conclusion

- The goal is to make the system **easy to extend** without incurring a high impact of change.
- This goal is accomplished by partitioning the system into **components**, and arranging those components into a **dependency hierarchy** that **protects** higher-level components from changes in lower-level components.

Liskov Substitution Principle (LSP)

Liskov Substitution Principle

- What is wanted here is something like the following substitution property:

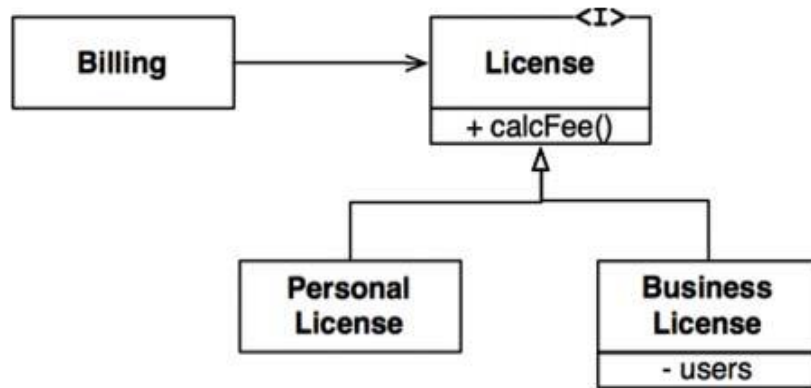
If for each object $o1$ of type S there is an object $o2$ of type T such that for all programs P defined in terms of T , the behavior of P is unchanged when $o1$ is substituted for $o2$ then S is a subtype of T .

- In other words:

If you can take any object of type **S** and use it anywhere an object of type **T** is expected **without breaking anything**, then **S is a subtype of T**.

LSP Example

- Imagine that we have a class named `License`, as shown in Figure. This class has a method named `calcFee()`, which is called by the *Billing* application.



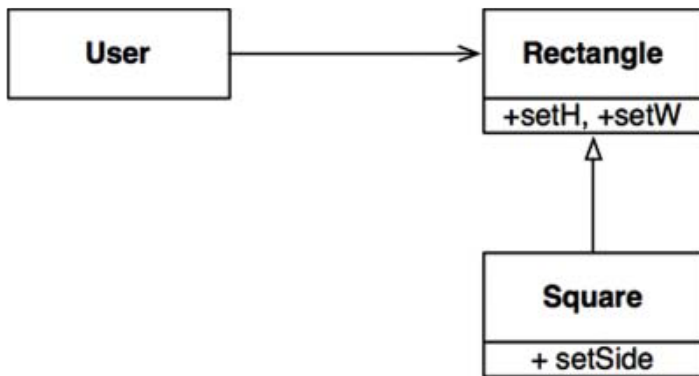
`License`, and its derivatives, conform to LSP

LSP Example (cont.)

- There are two “subtypes” of License: *PersonalLicense* and *BusinessLicense*. They use different algorithms to calculate the license fee.
- This design **conforms** to the LSP because the behavior of the Billing application does not depend, in any way, on which of the two subtypes it uses.

LSP Violation Example

- The canonical example of a violation of the LSP is the famed (or infamous, depending on your perspective) **square/rectangle problem**.



The infamous square/rectangle problem

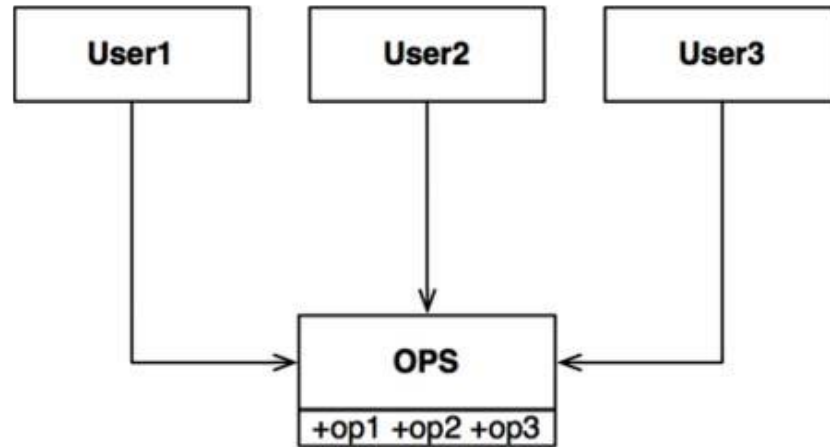
Conclusion

- In the early years of the object-oriented revolution, we thought of the LSP as a way to guide the **use of inheritance**, as shown in the previous sections.
- However, over the years the LSP has morphed into a broader principle of software design that pertains to **interfaces and implementations**.
- The LSP can, and should, be extended to the level of **architecture**.
A simple violation of substitutability, can cause a system's architecture to be polluted with a significant amount of extra mechanisms.

Interface Segregation Principle (ISP)

Interface Segregation Principle

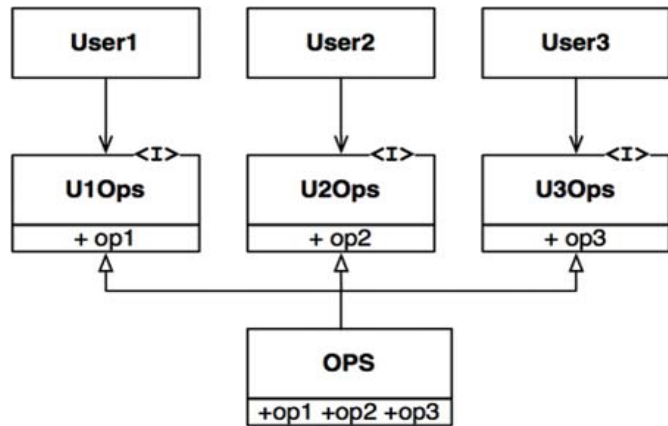
- The Interface Segregation Principle (ISP) derives its name from the diagram shown in Figure.



The Interface Segregation Principle

Interface Segregation Principle (cont.)

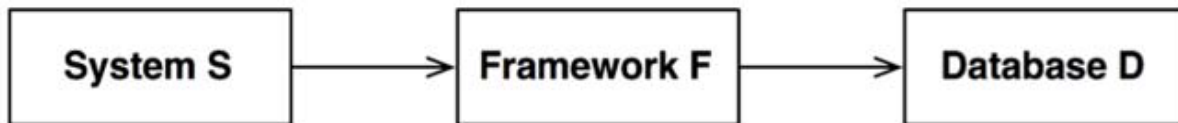
- The source code of *User1* will inadvertently depend on *op2* and *op3*, even though it doesn't call them.
- This dependence means that a change to the source code of *op2* in *OPS* will force *User1* to be **recompiled** and **redesployed**, even though nothing that it cared about has actually changed.



Segregated operations

ISP and Architecture

- In general, it is harmful to depend on modules that **contain more than you need**.
- Consider, for example, an architect working on a system, S. He wants to include a certain framework, F, into the system. Now suppose that the authors of F have bound it to a particular database, D. So, S depends on F (which depends on D).



A problematic architecture

Conclusion

- The lesson here is that depending on something that carries baggage that you don't need can cause you troubles that you didn't expect.

Dependency Inversion Principle (DIP)

Dependency Inversion Principle

- The most flexible systems are those in which source code dependencies refer only to **abstractions, not to concretions**.
- In a statically typed language, like Java, this means that the use, import, and include statements should refer only to source modules containing interfaces, abstract classes, or some other kind of abstract declaration.
- **Nothing concrete should be depended on.**

Dependency Inversion Principle (cont.)

- For these reasons, we tend to ignore the stable background of **operating system** and platform facilities when it comes to DIP.
- We tolerate those concrete dependencies because we know we can **rely on them** not to change.
- It is the *volatile* concrete elements of our system that we want to avoid depending on.
- Those are the modules that we are actively developing, and that are **undergoing frequent change**.

Dependency Inversion Principle (cont.)

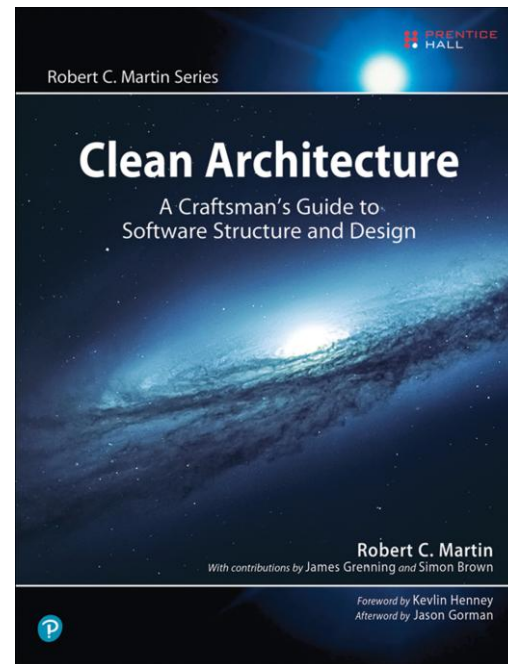
- *Interfaces are less volatile than implementations.*
- Good software designers and architects work hard to **reduce the volatility of interfaces**.
- Coding practices:
 1. Don't refer to volatile concrete classes. Refer to abstract interfaces instead.
 2. Don't derive from volatile concrete classes.
 3. Don't override concrete functions. You should make the function abstract and create multiple implementations.
 4. Never mention the name of anything concrete and volatile.

Conclusion

- DIP violations cannot be entirely removed, but they can be **gathered into a small number of concrete components** and kept separate from the rest of the system.
- Most systems will contain at least one such concrete component—often called ***main*** because it contains the main function.

Resources

- Clean Architecture (Robert C. Martin): Chapters 7, 8, 9, 10, and 11.



Q&A
